



WeCloudData

Data Transformation

Clean It, Shape It!

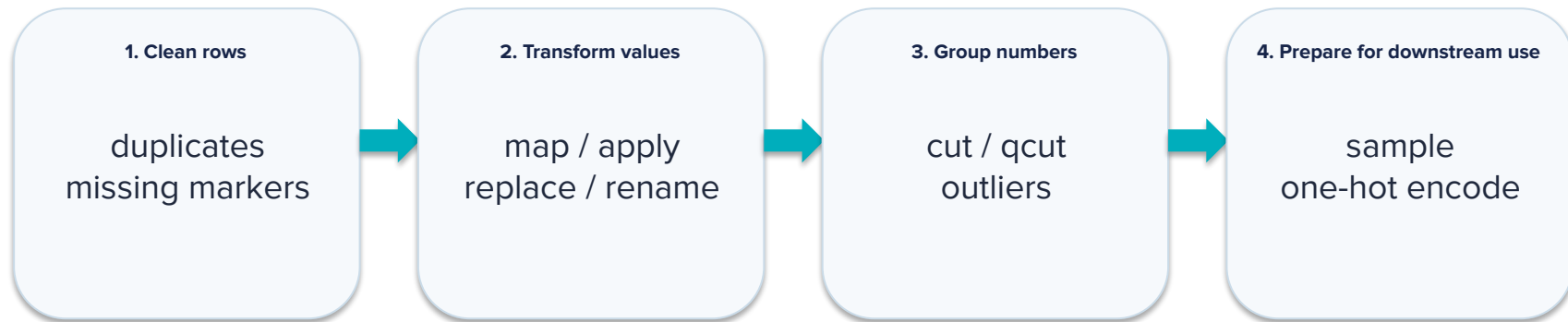




Data Transformation

Sample Workflow

Let's begin with a typical data transformation workflow.



Goal: make raw data clean, consistent, and ready for downstream use.



Data Transformation

What • Why • When

WHAT

A toolbox of pandas methods that change the values, labels, or shape of your data without rebuilding it from scratch.

WHY

Garbage in, garbage out. Clean, consistent, well-scaled data leads to trustworthy analysis and better ML models.

WHEN

Right after loading raw data and again whenever you spot duplicates, weird values, or columns needing encoding.

8 TECHNIQUES

Duplicates · map/apply · replace · rename · binning · outliers · sampling · dummy variables



Data Transformation

Messy room → tidy table

Messy data looks like this

Duplicate socks = repeated rows
Broken labels = inconsistent values
Wrong shelf names = unclear columns
Too many sizes = numbers need buckets



Transformation means

Keep one copy
Standardize values
Rename columns/index
Bin, filter, encode

The same pattern repeats: inspect first → transform deliberately → check the result.



WeCloudData

Removing Duplicates



Why duplicates matter

Why duplicates matter

A repeated row can make your analysis lie

row	name	age	purchase
0	Ana	25	\$20
1	Ben	30	\$50
2	Ana	25	\$20
3	Cleo	22	\$40

Before deleting, ask

Counts become inflated

Sums and averages shift

Model training may over-weight
repeated examples

Context decides whether two identical
rows are truly duplicates

'Ana, 25' twice could be a duplicate — or two different people. Always inspect first.



Find Duplicates

`df.duplicated()`

`duplicated()` — find them first

- Returns a **boolean Series**: True for rows that repeat an earlier row.
- By default, the **FIRST occurrence is marked as 'not duplicate'**.
- Before deleting, inspect the rows marked as duplicates using `df[df.duplicated()]`

```
df.duplicated()
# 0      False
# 1      False
# 2      True   <- same as row 0
# 3      False
```



Remove Duplicates

`drop_duplicates()`

`drop_duplicates()` — remove them

- `subset` lets you define duplicates by just one or a few columns.
- `keep='last'` keeps the last occurrence instead of the first.
- Useful when two rows share a customer ID but differ in timestamp — keep one record per customer.

```
df.drop_duplicates()  
  
df.drop_duplicates(  
    subset=['name'],  
    keep='last'  
)
```




List distinct values in one column

unique()

unique()

- Works on a single Series (one column) and returns distinct values as a NumPy array.
- Doesn't touch the whole DataFrame — just answers 'what categories exist here?'
- Prefer unique() when you only need to LIST distinct values in one column (e.g. which countries appear), not clean the table.

```
df['name'].unique()  
# array(['Ana', 'Ben',  
#       'Cleo'],  
#       dtype=object)
```



Quick Recap

`uplicated()` vs `drop_duplicates()` vs `unique()`

Need	Best method
Find duplicate rows	<code>df.duplicated()</code>
Remove duplicate rows	<code>df.drop_duplicates()</code>
List categories	<code>df["col"].unique()</code>



WeCloudData

Transform values with rules



Why transform values?

Turn messy values into consistent columns

1. Raw values are messy

city / price / text
may need
cleanup



2. Write the rule once

function /
replacement rule



3. Apply to column consistently

map / apply



Apply one rule to every value

Series.map()

Series.map() — value-by-value lookup

- map() applies a function OR a dictionary to **EACH element of a Series**.
- If a value isn't in your dictionary, map() returns NaN for it
- It is great for spotting unmapped categories!

```
city_to_region = {
    'Toronto': 'ON',
    'Vancouver': 'BC',
    'Montreal': 'QC'
}

df['region'] =
    df['city'].map(
        city_to_region)

df['city_lower'] =
    df['city'].map(
        lambda x: x.lower())
```



Series.map() vs the Old way

Series.map()



Series.map() — value-by-value lookup

[Series.map()] vs. The Old Way

PANDAS ITERATION: HOW WE FEEL

Using a ``for`` loop to iterate over each row in a DataFrame.

```
for index, row in df.iterrows():  
    new_column[index] = process_data(row['value'])
```



Using `Series.map()` to apply a function to a column.

```
df['new_column'] = df['value'].map(process_data)
```





Use `apply()` for custom logic

Works on one Series or across rows

`apply()` — more power, more flexibility

- Runs on a Series element-wise (like `map`, but can call full functions), OR on a DataFrame row-wise / column-wise via `axis`.
- `map` = simple substitution table or single-value function on a Series.
- `apply` = more complex logic, can work across whole rows or columns.

```
# Series apply
df['age_group'] =
    df['age'].apply(
        lambda x:
            'adult' if x>=18
            else 'minor')

# Row-wise apply
df['full_name'] =
    df.apply(lambda r:
        r['first']+' '+
        r['last'], axis=1)
```



Speed tip

Vectorize when you can

```
# slower: Python function per row
df['price_tax'] = df['price'].apply(
    lambda x: x * 1.13
)
```



```
# faster: pandas/NumPy vectorized
operation
df['price_tax'] = df['price'] *
1.13
```

Rule of thumb: map/apply are fine for small data and custom logic; vectorized code is better for large tables.



map vs apply

Choose based on the kind of rule

Situation	Use
Dictionary lookup	<code>map()</code>
Simple value-level function	<code>map()</code> or <code>apply()</code>
Logic across multiple columns	<code>apply(axis=1)</code>
Simple math on whole column	vectorized operation

Avoid using `apply` just because it works. Ask whether pandas can do the operation directly on the whole column.



WeCloudData

Replacing Values



Why replace values?

Standardize messy markers before downstream use

1. Raw markers vary

'Missing' · 'N/A' · '-'
'?' · 999 · -1

2. Convert to one meaning

Use a recognized missing marker:
np.nan

3. Let pandas handle it

isna() · dropna()
fillna() · models

Key idea: replace inconsistent values before they break analysis or model training.



replace() swaps exact values

Single value, list, or dictionary replacement

Three common patterns

- Single old value → one new value
- Many old values → one new value
- Dictionary → different replacements
- Most pandas methods return a new object, so reassign the result.

Use replace() when you know the exact value(s) you want to swap.

```
import numpy as np

# one value -> one replacement
df['status'] = df['status'].replace(
    'Missing', np.nan
)

# many values -> one replacement
df['status'] = df['status'].replace(
    ['Missing', 'N/A', '?'],
    np.nan
)

# dictionary -> different replacements
df['status'] = df['status'].replace({
    'Y': 'Yes',
    'N': 'No'
})
```



Use `.loc[]` for conditional changes

Boolean masks are for rules, not exact value swaps

Choose the tool by the rule

- `replace()` is best for exact known values.
- `.loc[mask, col] = value` changes rows that meet a condition.
- Use it for ranges, invalid values, or threshold-based cleanup.

Exact match → `replace()`; condition or range
→ `.loc[]`.

```
# cap negative ages at 0
df.loc[df['age'] < 0, 'age'] = 0

# values above 100 -> missing
df.loc[df['age'] > 100, 'age'] =
np.nan
```



WeCloudData

Renaming Axis Indexes



Rename columns clearly

Use readable labels without changing the values

Two ways to rename columns

- `df.columns = [...]` replaces every column name.
- `rename(columns={...})` changes only selected columns.
- Selected rename is safer when you only need targeted cleanup.

Renaming changes labels, not the actual data inside the columns.

```
# Option 1: replace all names
df.columns = [
    'id', 'full_name', 'age_years'
]

# Option 2: rename selected columns
df = df.rename(columns={
    'name': 'full_name',
    'age': 'age_years'
})
```



Rename index labels

Index labels identify rows; column names identify variables

Index operations are different

- `rename(index={...})` relabels selected rows.
- `df.index.name` gives the index itself a name.
- `df.index.map(func)` transforms every index label.

Use index renaming only when the row labels carry meaning.

```
# relabel selected index values
df = df.rename(index={
    0: 'first_row',
    1: 'second_row'
})

# name the index itself
df.index.name = 'customer_id'

# transform every index label
df.index = df.index.map(str.upper)
```




WeCloudData

Discretization & Binning



Why bin numbers?

Turn exact values into useful groups

**1. Exact numbers
can be too detailed**

Age 23, 24, and 25
may all mean
“young adult”.

**2. Bins create
categories**

Group values into
labels such as
Teen, Adult, or
Senior.

**3. Patterns become
easier to read**

Useful for reporting,
visualization, and
model features.

Binning reduces detail when categories are more useful than exact values.



cut() uses known bin edges

Best for domain-defined groups such as age ranges or risk levels

Key pieces

- bins are interval edges.
- There is always one more edge than label.
- labels name the groups.
- Intervals are right-closed by default: (12, 19].

Use cut() when the boundaries come from business rules or domain knowledge.

```
ages = [5, 17, 22, 35, 45, 67, 90]
bins = [0, 12, 19, 35, 60, 100]
labels = [
    'Child', 'Teen', 'Young Adult',
    'Adult', 'Senior'
]

pd.cut(ages, bins=bins, labels=labels)
# Child, Teen, Young Adult, ...
```



cut() vs qcut()

Choose fixed boundaries or roughly equal-sized groups

Decision rule

- Use cut() when the bin edges are meaningful before looking at the data.
- Use qcut() when each group should contain a similar number of rows.
- Both create categories, but the boundary logic is different.

Business-defined edges → cut(); **data-driven quantile groups** → qcut().

```
# Fixed age ranges
pd.cut(
    ages,
    bins=[0, 12, 19, 35, 60, 100]
)

# Four roughly equal-sized groups
pd.qcut(
    df['score'],
    q=4,
    labels=['Q1', 'Q2', 'Q3', 'Q4']
)
```



WeCloudData

Detecting & Filtering Outliers



IQR flags outliers

Robust for skewed data because it uses quartiles

How it works

- Find Q1 and Q3.
- Compute $IQR = Q3 - Q1$.
- Flag values below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$.
- Inspect flagged rows before removing anything.

Outliers are flags for review; they are not always errors.

```
Q1 = df['value'].quantile(0.25)
Q3 = df['value'].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = df[
    (df['value'] < lower) |
    (df['value'] > upper)
]
```



Z-score flags distant values

Best when the distribution is roughly normal

Z-score idea

- $z = (x - \text{mean}) / \text{standard deviation}$.
- A common rule is to flag $|z| > 3$.
- Works best when the distribution is roughly bell-shaped.
- Use IQR instead when the data is strongly skewed.

Choose the outlier method based on the shape of the data.

```
z = (df['value'] -  
     df['value'].mean()) /  
     df['value'].std()  
  
outliers = df[z.abs() > 3]  
  
clean_df = df[z.abs() <= 3]
```



sample() shuffles or selects rows

Useful for downstream checks and train/test preparation

Why sampling helps

- Shuffle rows to reduce ordering bias before splits.
- Inspect a small subset when the table is large.
- random_state makes the random sample reproducible.
- Use n for a count or frac for a percentage.

Sampling is usually a preparation or checking step, not an outlier method.

```
# shuffle all rows
df = df.sample(
    frac=1,
    random_state=42
).reset_index(drop=True)

# random 5 rows
df.sample(n=5, random_state=42)

# random 10%
df.sample(frac=0.1, random_state=42)
```




One-hot encode categories

Turn text labels into 0/1 indicator variables

Why encode categories?

- Many ML models need numeric input.
- Each category becomes a new 0/1 column.
- A row gets 1 when that category is present, otherwise 0.
- Use `get_dummies()` for a quick pandas encoding.

One-hot encoding is a common final step before feeding categorical features into ML.

```
df = pd.DataFrame({
    'color': ['Red', 'Blue', 'Green',
             'Blue']
})

pd.get_dummies(df, columns=['color'])
```

	color_Blue	color_Green	color_Red
#	0	0	1
#	1	0	0
#	0	1	0
#	1	0	0



Data transformation checklist

Put the methods back into the workflow

1. Clean rows

Find duplicates
`uplicated()`

Remove duplicates
`drop_duplicates()`

List categories
`unique()`

2. Transform values

Map values
`map()`

Custom logic
`apply()`

Replace values
`replace() / .loc[]`

Rename labels
`rename()`

3. Group numbers

Fixed bins
`cut()`

Quantile bins
`qcut()`

Flag outliers
IQR / Z-score

4. Prepare downstream

Shuffle or inspect
`sample()`

Encode categories
`get_dummies()`

Workflow mindset: inspect first, transform deliberately, then check the result.



Putting it all together

CHEAT SHEET

Task	Method
Find duplicate rows	<code>df.duplicated()</code>
Remove duplicate rows	<code>df.drop_duplicates(subset=..., keep='first')</code>
Distinct values in a column	<code>df['col'].unique()</code>
Transform a column	<code>df['col'].map({...})</code> or <code>.apply(func)</code>
Replace values	<code>df['col'].replace(old, new)</code>
Replace by condition	<code>df.loc[mask, 'col'] = new_value</code>
Rename columns / index	<code>df.rename(columns={...}, index={...})</code>
Bin by custom edges	<code>pd.cut(data, bins=[...], labels=[...])</code>
Bin by quantiles	<code>pd.qcut(data, q=4)</code>
Outliers (IQR)	<code>Q1, Q3 = quantile(.25/.75); IQR = Q3-Q1</code>
Outliers (Z-score)	<code>(x-mean)/std, flag z > 3</code>
Shuffle rows	<code>df.sample(frac=1, random_state=...)</code>
One-hot encode	<code>pd.get_dummies(df, columns=[...], drop_first=True)</code>